

第三周周报

一、引言

第一月的课程按“几何表示 → 运动生成 → 策略学习”三层推进：第 1 周讲机器人的空间描述语言，第 2 周讲怎么让机器人从起点走到终点，第 3 周讲怎么让机器人自己学会这件事。本文按这三层依次整理理论要点，并回答课程布置的全部 9 道思考题。

三块内容并不是彼此独立的，而是同一个问题的三次抽象：

- **坐标系与位姿变换**回答“机器人在哪、朝哪”——它是后两者的共同语言，任何轨迹和任何策略最终都要落成一组位姿序列。
- **轨迹规划**回答“怎么从当前位姿走到目标位姿”——在已知模型的前提下，把几何路径和动力学约束解耦处理。
- **深度强化学习**回答“模型未知时怎么办”——把位姿和动作当作状态与动作，用交互数据直接学一个策略，从而绕过显式建模与显式规划。

本文只讨论理论。每一处结论都给出推导或判据，所用的算例都是纯代数的，不依赖任何实验结果；与之配套的代码实现与实测数据见《复现代码与结果说明》。

二、坐标系与位姿变换

2.1 为什么需要一个整体的齐次矩阵来表示位姿（作业题 1）

如果分开用“旋转 R 加平移 t ”描述位姿，把 A 系下的点变到 B 系要写两步：

$$p' = Rp + t \quad (1)$$

这个写法本身没问题，麻烦出在**连续变换**上。设从坐标系 A 到 C 要经过中间系 B ，分开写就变成“先旋转再平移，再旋转再平移”的嵌套：

$$p_C = R_{BC}(R_{AB}p_A + t_{AB}) + t_{BC} = (R_{BC}R_{AB})p_A + (R_{BC}t_{AB} + t_{BC}) \quad (2)$$

每多一级链条，就多一次矩阵乘、一次向量加，而且必须手工维护“旋转部分相乘、平移部分先旋转再相加”这条容易记错的合成规则。更麻烦的是求逆：要对 $p' = Rp + t$ 求反向映射，得先解 $Rp = p' - t$ ，再对 R 求逆，两步都容易写错。

齐次坐标的思路是把 n 维仿射变换写成 $n+1$ 维的线性变换，于是“旋转 + 平移”被压进同一次矩阵乘法：

$$\mathbf{T} = \begin{bmatrix} R & p \\ 0 & 1 \end{bmatrix} \quad (3)$$

其中右下角的 1 与左下角的 0 是占位项，用来把仿射变换补成线性变换。引入齐次坐标后，作用到点上也只需一次乘法：

$$\tilde{p}' = \mathbf{T}\tilde{p}, \quad \tilde{p} = (p^\top, 1)^\top \quad (4)$$

整条链因此变成一串矩阵相乘，形式统一、没有特例：

$$\mathbf{T}_{AC} = \mathbf{T}_{AB}\mathbf{T}_{BC}, \quad \mathbf{T}_{0n} = \mathbf{T}_{01}\mathbf{T}_{12} \cdots \mathbf{T}_{n-1,n} \quad (5)$$

用一个整体矩阵除了“写法统一”，还换来三条实质性的好处：

- **求逆有闭式解。**旋转矩阵满足 $R^\top R = I$ 且 $\det R = +1$ ，即正交阵，所以逆矩阵可以直接写出，不需要数值求逆：

$$\mathbf{T}^{-1} = \begin{bmatrix} R^\top & -R^\top p \\ 0 & 1 \end{bmatrix} \quad (6)$$

这一点在实时控制里很重要：数值求逆是 $O(n^3)$ 且要处理条件数，而解析逆只是转置和一次矩阵向量乘，还天然数值稳定。

- **复合满足结合律。** $(T_1 T_2) T_3 = T_1 (T_2 T_3)$ ，所以长链条可以任意分段缓存。机械臂控制里“算一次基座到腕部的变换，之后每个周期复用”就靠这个性质。
- **退化情形被统一覆盖。** 纯旋转是 $p = 0$ ，纯平移是 $R = I$ ，不需要为它们各写一套分支；而在分开表示的写法里，“没有旋转”意味着绕某轴转 0° ，“没有平移”意味着 $t = 0$ ，两套特例都要单独判。

从群论的角度看，刚体变换的集合构成特殊欧氏群 $SE(3)$ ，齐次矩阵正是它在 4×4 矩阵下的忠实表示（同态且单射）。所以“用矩阵表示位姿”不是记号上的方便，而是把刚体运动嵌入线性代数，从而使复合、求逆、作用到点这些运算全部成为标准的矩阵运算。

2.2 点在 A 坐标系下，如何换算到 B 坐标系（作业题 2）

结论是：**用坐标系变换矩阵左乘坐标向量**。但方向约定是最容易出错的地方，所以先把记号钉死，全文统一： T_{AB} 表示“B 系在 A 系中的位姿”。

既然 T_{AB} 描述的是 B 系在 A 系中的位置与朝向，它天然就把 B 系的坐标解释到 A 系里：

$$p_A = T_{AB} p_B \quad (7)$$

反过来，要把 A 系下的点换算到 B 系，就在上式两边左乘 T_{AB} 的逆。由 2.1 的闭式解，这个逆恰好就是 T_{BA} ：

$$p_B = T_{AB}^{-1} p_A = T_{BA} p_A \quad (8)$$

这个等式说明了记号的**自治性**： T_{AB} 的逆就是交换下标的 T_{BA} ，不需要额外记忆任何规则。写代码时只要坚持“下标从右往左读”，即 T_{AB} 把 B 的量变成 A 的量，就不会搞反方向。

对点与对向量作用不同。 同样一个 4×4 矩阵，作用到“点”和“方向向量”上结果不同：点带位置信息，齐次坐标末位为 1，平移会生效；方向向量（如速度、法向、姿态轴）不该被平移，齐次坐标末位应取 0，此时 T 对它的作用只保留旋转部分。这是几何上“仿射变换”与“线性变换”的区别，混用会造成“速度被平移污染”这类隐蔽错误。

顺序不可交换。 一般地 $T_1 T_2 \neq T_2 T_1$ ，因为“先转后移”和“先移后转”得到的是两个不同的刚体运动。只有当两者的旋转轴/平移方向满足特殊关系（例如绕同一轴的两个旋转、或沿同一方向的两个平移）时才可交换。所以链式相乘的顺序必须与运动学链的物理顺序严格一致。

两种链式写法要分清。 求整体矩阵是**右乘** ($T_{0n} = T_{01} T_{12} \cdots T_{(n-1),n}$)，把矩阵作用到点是**左乘** ($p_0 = T_{0n} p_n$)，而“沿链条逐级把点往前推”是**从末端向基座**依次左乘各段的矩阵。这三件事说的其实是一回事，但写成文字很容易混淆，稳妥的做法是始终用 $p_i = T_{ij} p_j$ 这一个式子去推。

2.3 多关节坐标系之间的变换如何得到末端相对基座的位姿（作业题 3）

答案就是 2.1 的链式法则在关节链上的具体化：**把相邻坐标系的变换连续右乘。**

要让这件事可算，需要一套统一的参数化方式，把任意两个相邻连杆坐标系之间的刚体变换用最少数目写出来。Denavit–Hartenberg (DH) 约定给出的答案是四个参数：连杆长度 a 、连杆扭角 α 、关节偏置 d 、关节角 θ ，其中只有一个是变量。标准 DH 把变换写成四个基本变换的固定顺序之积：

$$\mathbf{T}_{i-1,i} = R_z(\theta_i) T_z(d_i) T_x(a_i) R_x(\alpha_i) \quad (9)$$

于是从基座（系 0）一路乘到末端（系 n ）：

$$\mathbf{T}_{0n} = \mathbf{T}_{01} \mathbf{T}_{12} \cdots \mathbf{T}_{n-1,n} \quad (10)$$

展开时有一处极易出错的地方：平移列会被 $R_z(\theta)$ 带走。把上式展开可以看到， $T_z(d) T_x(a) R_x(\alpha)$ 的平移部分是 $(a \ 0 \ d)$ ，但它左边还乘了一个 $R_z(\theta)$ ，所以 $T_{(i-1),i}$ 的平移列实际是

$$p_{i-1,i} = R_z(\theta_i) (a_i, 0, d_i)^\top \quad (11)$$

它是随关节角变化的。把平移列误写成与 θ 无关的常量，单关节看起来“差不多”，但整条链会立刻错位——而且这种错误不会报错，只会让末端跑到错误的位置。

另外两个常见约定问题：

- **标准 DH 与改进 DH 不同。**标准 DH 把关节变量放在最左边（先转后移），改进（Craig）DH 把关节变量放在中间。两者都是合法的，但参数表不可混用；拿到一份 DH 表先确认它是哪一种。
- **角度偏置与零位。**实际机器人都有机械零位与 DH 零位的偏差，以及关节转向与正方向的符号约定，这些都要作为附加偏置写进参数表，否则会出现“关节转反”或“零位不对”的现象。

链式的另一面：求逆没有闭式解。正运动学 T_{0n} 是各关节角的显式函数，计算是 $O(n)$ 的矩阵连乘；而给定末端的 T_{0n} 反求关节角（逆运动学）是一个非线性方程组，一般有多个解甚至无穷多解（冗余机械臂）。这解释了为什么规划通常在关节空间或笛卡尔空间各自展开，以及为什么要有“解的选择”策略（最靠近当前位形的解、避开奇异与限位的解等）。

2.4 姿态的表示与奇异：欧拉角、四元数、SLERP

位置是三维向量，没有歧义；姿态则有多种互相等价的表示，各有各的代价。理解这些代价是理解“为什么真实系统最终多用四元数”的前提。

旋转矩阵。 3×3 ，九个数但只有三个自由度（正交性给出六个约束）。优点是复合即矩阵乘法、作用到向量即矩阵向量乘、无奇异；缺点是冗余、数值上会缓慢失去正交性需要重新正交化，而且不能直接做插值（逐元素插值出来的矩阵不再是旋转矩阵）。

欧拉角。三个角度，最直观，人机界面和小角度描述都爱用它。以常用的 ZYX (roll-pitch-yaw) 约定为例，复合顺序为

$$R = R_z(\psi) R_y(\theta) R_x(\phi) \quad (12)$$

它的致命问题是**万向锁**。把 pitch 取到 $+90^\circ$ ，中间那个 $R_y(90^\circ)$ 会把 x 轴转到 $-z$ 、把 z 轴转到 $+x$ ，于是最外层绕 z 的旋转与最内层绕 x 的旋转变成绕同一条轴，两者不再可分辨。代数上可以严格验证：

$$R_z(\psi) R_y(90^\circ) R_x(\phi) = R_z(\psi - \phi) R_y(90^\circ) \quad (13)$$

也就是说，此时只有“偏航减横滚” $(\psi - \phi)$ 这一个组合量可观测，而 ψ 与 ϕ 各自的值无法从 R 中恢复——自由度从三降到了二。这正是万向锁的数学本质：**不是数值退化，而是参数化本身在该点降秩**。表现上就是逆解在 $\text{pitch} \rightarrow \pm 90^\circ$ 时误差急剧放大，且插值会出现突变。

单位四元数。四个数（一个约束）， $q = (w, x, y, z)$ ，其中 $w = \cos(\theta/2)$ ， $(x, y, z) = \sin(\theta/2) \cdot \hat{n}$ ， $\hat{n}$ 是转轴。它的复合是四元数乘法，作用到向量可写成旋转公式，全程无奇异，而且只用一个约束做归一化即可，数值上比 3×3 矩阵稳定得多。代价是存在**双覆盖**： q 与 $-q$ 表示同一个旋转，这本身不是问题（大不了一倍冗余），但插值和求距离时必须显式处理，否则会走上绕远的那条弧。

姿态插值：为什么必须用 SLERP。给定起止姿态 q_0 、 q_1 和一个归一化时间 t ，球面线性插值 (SLERP) 的定义是

$$\text{slerp}(q_0, q_1, t) = \frac{\sin((1-t)\Omega)}{\sin \Omega} q_0 + \frac{\sin(t\Omega)}{\sin \Omega} q_1, \quad \Omega = \arccos(q_0 \cdot q_1) \quad (14)$$

它有两个关键性质。第一，**角速度恒定**：插值路径是单位球面上连接 q_0 与 q_1 的最短弧（测地线），转角关于 t 是线性的，因此末端转动是匀速的。第二，**必须取短弧**：由于双覆盖，若 $q_0 \cdot q_1 < 0$ 应先把 q_1 取负，否则会沿大圆绕远路，出现“转了 350° 而不是 10° ”的现象。

作为对比，对欧拉角三个分量分别做线性插值，得到的路径一般**不是**测地线：三个角各自匀速变化时，合成转动的角速度并不恒定；在接近万向锁时还会出现剧烈抖动。所以在笛卡尔空间规划里，位置用线性插值、姿态用 SLERP（或等价的旋转向量插值），两者再共用同一条时间标度，是标准做法。

三、轨迹规划

3.1 轨迹规划时为什么要限制速度和加速度（作业题 1）

可以从物理、机械、控制、系统四个层面回答，前两层是硬约束，后两层是工程原因。

第一层：执行器能力。关节力矩与运动满足刚体动力学方程

$$\tau = M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) \quad (15)$$

其中 $M(q)\ddot{q}$ 是惯性力矩项。加速度越大，这一项越大；速度越大，科氏/离心项 $C(q, \dot{q})\dot{q}$ 也越大，同时电机反电动势升高、可用转矩下降。两者叠加会导致驱动器过流保护或转矩饱和，结果不是“精度差一点”，而是直接报故障停机。

第二层：机械冲击与寿命。加速度的导数称为加加速度（jerk, $j = \ddot{\dot{q}}$ ）。加速度不连续意味着 jerk 出现冲激，它对应减速器齿轮上的冲击载荷。工程上限制加速度，本质上是在限制 jerk 的峰值，从而延长减速器寿命、减小振动与噪声。这也是为什么高精度场合在梯形剖面之上还要再叠一层 jerk 受限的 S 型曲线。

第三层：可跟踪性。控制器带宽有限。若指令的速度/加速度超出跟踪能力，实际轨迹会滞后于规划轨迹，误差随加速度增大而增大。在协作机器人这类需要维持安全距离的场景里，“实际轨迹偏离规划轨迹”意味着安全评估失效，属于安全问题而非性能问题。

第四层：系统的可判定性。给定了 $v_{\max}$ 与 $a_{\max}$ ，一段位移的最短耗时可以被解析地算出来（见 3.3）。上层调度只有在这个时长确定的前提下，才敢做节拍规划与多机协同。反过来，若不对速度加速度设限，轨迹时长就没有下界，系统无法调度。

最关键的一点：只约束端点是远远不够的。多项式插值通常只给定起止位置（必要时加上起止速度、加速度），而中间段的峰值完全由插值阶数和时长决定。以最常用的三次多项式为例，取 $q(0) = q_0$ 、 $q(T) = q_1$ 、两端速度为零，可解得（记 $\Delta q = q_1 - q_0$ ）：

$$q(t) = q_0 + \frac{3\Delta q}{T^2}t^2 - \frac{2\Delta q}{T^3}t^3 \quad (16)$$

对它求导、再求导，并取极值点（速度的极值点在 $t = T/2$ ，加速度的极值点在两端 $t = 0$ 与 $t = T$ ），得到中间段的峰值闭式解：

$$\max |\dot{q}| = \frac{3\Delta q}{2T}, \quad \max |\ddot{q}| = \frac{6\Delta q}{T^2} \quad (17)$$

这两个式子把问题说清楚了：**峰值与 Δq 成正比、与 T 成反比（加速度反比于 T^2 ）**。所以缩短时长会让峰值迅速上升。举一个纯代数的算例：取 $\Delta q = 1 \text{ rad}$ 、 $T = 1 \text{ s}$ ，则峰值速度为 1.5 rad/s 、峰值加速度为 6 rad/s^2 ；若系统上限是 $v_{\max} = 0.6 \text{ rad/s}$ 、 $a_{\max} = 2.0 \text{ rad/s}^2$ ，这条

曲线**两项都超限**（分别超 2.5 倍与 3 倍）。也就是说，只给端点条件的规划结果在物理上不可执行，必须把速度与加速度作为显式约束写进规划器。

3.2 多项式插值的数学：三次与五次

关节空间的点对点运动最常用多项式，因为它求解简单、光滑性可控。关键认识是：**多项式的阶数由边界条件的个数决定**。

三次多项式有四个系数，恰好对应四个边界条件：起点位置、终点位置、起点速度、终点速度。点对点运动通常取两端速度为零，于是得到 3.1 中那条曲线，系数为 $a_0 = q_0$ 、 $a_1 = 0$ 、 $a_2 = 3\Delta q/T^2$ 、 $a_3 = -2\Delta q/T^3$ 。它的加速度在两端不连续（从 0 跳到 $6\Delta q/T^2$ ），所以 jerk 有冲激。

五次多项式有六个系数，因此可以额外指定两端加速度：

$$q(t) = c_0 + c_1 t + c_2 t^2 + c_3 t^3 + c_4 t^4 + c_5 t^5 \quad (18)$$

六个系数由六个线性方程确定——对应两端的位置、速度、加速度：

$$q(0) = q_0, \quad \dot{q}(0) = v_0, \quad \ddot{q}(0) = a_0, \quad q(T) = q_1, \quad \dot{q}(T) = v_1, \quad \ddot{q}(T) = a_1 \quad (19)$$

由于两端加速度可指定为零，五次多项式的加速度在端点连续，jerk 不再有冲激，所以它比三次多项式平滑得多。代价是可能出现的超调（中间段冲过终点再退回来），以及峰值更难闭式控制。

两者的共同短板：无论三次还是五次，峰值都只在端点被约束，中间段不受控。要真正保证不超限，有两条路径：一是把峰值约束作为额外条件去解更高阶的多项式或做时间重参数化；二是换用下面这种把限制直接写进结构的剖面。

3.3 梯形速度剖面与固定时长的可行性判据（作业题 2）

梯形速度剖面（又称 LSPB, Linear Segments with Parabolic Blends）把运动固定分成三段：匀加速 → 匀速 → 匀减速。由于每段的速度或加速度都是常数，峰值自然被钳在给定上限内，不需要事后检查。设位移 Δq 、速度上限 $v_{\max}$ 、加速度上限 $a_{\max}$ ，则加速段时长为

$$t_a = \frac{v_{\max}}{a_{\max}} \quad (20)$$

若位移足够长（ $\Delta q \geq v_{\max} \cdot t_a$ ），剖面是梯形，匀速段时长 $t_f = (\Delta q - v_{\max} t_a)/v_{\max}$ ，总时长

$$T_{\text{LSPB}} = 2t_a + t_f = \frac{\Delta q}{v_{\max}} + \frac{v_{\max}}{a_{\max}} \quad (21)$$

若位移太短、来不及加速到 $v_{\max}$ ，剖面退化为三角形（匀速段为零），此时峰值速度由 $a_{\max}$ 决定： $v_{\text{peak}} = \sqrt{\Delta q a_{\max}}$ ，总时长 $T = 2\sqrt{\Delta q/a_{\max}}$ 。

现在回到“规定 T 秒内必须完成”这个问题。它可以拆成三步推理。

第一步：先算一个跨不过去的下界。无论怎么规划，平均速度都是 $\Delta q/T$ ；而峰值速度一定不低于平均速度，所以

$$v_{\text{peak}} \geq \frac{\Delta q}{T} \quad (22)$$

若这个下界已经超过 v_{max} ，那么不用细算，直接判定不可行。

第二步：在速度与加速度之间找平衡点。设允许的加速时间为 t_a ，则峰值速度为 $\Delta q/(T - t_a)$ 、峰值加速度为 v_{peak}/t_a 。注意这两个量随 t_a 的变化方向相反： t_a 越大，峰值速度越低，但峰值加速度越高； $t_a \rightarrow 0$ 时加速度发散， $t_a \rightarrow T/2$ 时退化为三角形剖面。因此在 $t_a \in (0, T/2]$ 上， v/v_{max} 单调递增、 a/a_{max} 单调递减，要使两者中的较大者最小，最优点就在两条曲线的交点：

$$\frac{v}{v_{\text{max}}} = \frac{a}{a_{\text{max}}} \implies t_a^* = \frac{v_{\text{max}}}{a_{\text{max}}} \quad (23)$$

这个结果很有意思：**给定总时长时的最优加速时间，与自然 LSPB 的加速时间完全相同。**也就是说，最优策略不是“把梯形压扁”，而是保持梯形形状、整体缩放。

第三步：用判据做决策。把 t_a^* 代回可行性条件 $v_{\text{peak}} \leq v_{\text{max}}$ ，得到一条非常干净的等价判据：

$$T \geq \frac{\Delta q}{v_{\text{max}}} + \frac{v_{\text{max}}}{a_{\text{max}}} = T_{\text{LSPB}} \quad (24)$$

即“T 秒内能否完成”等价于“自然 LSPB 的总时长是否不超过 T”。这解释了 3.1 第四层说的“时长可以提前算出来”：它就是一个关于 Δq 的仿射函数。下面给两个纯代数的算例（取 $v_{\text{max}} = 0.6 \text{ rad/s}$ 、 $a_{\text{max}} = 2.0 \text{ rad/s}^2$ 、 $T = 2 \text{ s}$ ）：

行程 Δq	自然时长 T_{LSPB}	最优峰值速度	最优峰值加速度	能否在 2 s 内完成
0.8 rad	$0.8/0.6 + 0.3 = 1.633 \text{ s}$	0.471 rad/s ($0.78 \times v_{\text{max}}$)	1.569 rad/s^2 ($0.78 \times a_{\text{max}}$)	可以
1.2 rad	$1.2/0.6 + 0.3 = 2.300 \text{ s}$	0.706 rad/s ($1.18 \times v_{\text{max}}$)	2.353 rad/s^2 ($1.18 \times a_{\text{max}}$)	不可以

表 1：两秒约束下的可行性算例。不可行时两个限制同时超出 18%，说明速度与加速度必须成比例地一起放宽，单独放松其中一个无效。

所以“规定两秒完成”的正确处理不是硬压一条三次多项式上去，而是：先按上式算出自然时长，若短于 2 s 就留出余量（提前到位后保持静止，便于对齐节拍）；若长于 2 s，则说明在这组限制下物理上做不到，此时只有三个选项——放宽 $v_{\text{max}}/a_{\text{max}}$ 、把动作拆成多段（每段各自满

足限制)、或者延长节拍。表 1 中 1.2 rad 的例子两个限制同时超 18%，正是因为最优点上它们被同时顶满，只放松一个没有意义。

3.4 笛卡尔空间轨迹与关节空间轨迹

同一段起止位姿，在两种空间里插值会得到完全不同的末端路径，核心差别在于“线性”这个词作用在哪个坐标系里。

关节空间插值。对关节角向量直接在起点与终点之间线性插值： $q(t) = (1 - t)q_0 + t q_1$ 。它的优点是每个关节的运动都单调、不会跳变，计算量极小（不需要反复求逆解），而且天然不会碰到运动学奇异（因为压根没经过笛卡尔空间）。缺点是**末端路径一般不是直线**：末端位置是关节角的非线性函数（正运动学），关节空间里的直线经过非线性映射后，在笛卡尔空间里是一条曲线。

笛卡尔空间插值。先对末端位置线性插值、姿态做 SLERP，再用逆运动学逐点求出对应的关节角。位置线性插值保证末端严格走直线段，姿态 SLERP 保证角速度恒定。代价是需要频繁求解逆运动学（可能有解不存在、多解、接近奇异等问题），而且求出的关节角序列可能出现跳变（相邻解不连续），必须做解的选择与连续性处理。

两者的取舍原则是：**对末端轨迹形状有要求的工艺（焊接、涂胶、切割）必须走笛卡尔空间；只关心“到位”的点对点搬运，走关节空间更快更稳。**工程上常见的折中是在笛卡尔空间生成关键路径点，再在关节空间对关键点之间做多项式插值（即轨迹由多段组成），兼顾形状与平顺性。

还有一个容易忽视的点：**关节空间插值不保证末端速度上限。**由于 $J(q)$ 随位形变化，关节匀速运动对应的末端速度会变化；在接近奇异位形时，很小的关节速度就可能对应极大的末端速度（或反之，末端很小的速度需要极大的关节速度）。所以关节空间的轨迹必须换算到约束更紧的一侧去检查。

3.5 采样法路径规划：RRT 与 PRM

当障碍物复杂、无法用解析式描述时，用**随机采样**把连续空间离散成一张图或一棵树，再在图上搜索。这类方法只要求“能判断一个位形是否碰撞、一段路径是否碰撞”，不需要障碍物的解析表达，通用性极强。

RRT（快速扩展随机树）。从起点长出一棵树，每次迭代：随机采一个位形 q_{rand} → 找树上离它最近的节点 → 朝它方向走固定步长得到 q_{new} → 若这段无碰撞就把它加入树。重复直到树接近目标。它的优点是**概率完备**（只要存在可行路径，采样足够多时以概率 1 找到），而且对高维空间和窄通道都有不错的适应性；缺点是路径很绕（步长固定、随机性强），不保证最优。

RRT*：把“渐进最优”加进来。它对 RRT 做两处修改：

- **choose parent（选父节点）：**新节点 q_{new} 不从“最近邻”直接挂树，而是在一个半径 r 的邻域内，挑一个“从起点到 q_{new} 累计代价最小”的节点当父节点。
- **rewire（重连）：**再看一遍邻域内的已有节点，如果它们经由 q_{new} 走能更省，就把它们的父

节点改成 q_{new} ，并更新它们的累计代价。

这两步使 RRT* 具有**渐进最优性**：采样数趋于无穷时，解以概率 1 收敛到最优。代价是每次迭代要多次做邻域查询与碰撞检测，单步更慢。RRT、RRT* 都是单查询（single-query）方法，适合“当前这一次”从 A 到 B 的规划。

PRM（概率路线图）。思路相反：先在自由空间里随机撒点，把彼此可见（连线无碰撞）的点连成一张无向图（路线图），查询时把起点终点接入图，再跑一次最短路搜索。它把计算量花在**预处理**上，因此适合**多查询**场景（同一环境下反复规划，例如仓储机器人）。但环境一旦变化，路线图就要重建，动态环境不适用。

采样之后的两个收尾步骤。采样法给出的原始解是折线，直接执行会走得很别扭，所以通常接两步后处理：

- **捷径平滑（shortcut）**。随机取路径上的两个点，若它们之间能直连无碰撞，就把中间那段折线替换成直线段。这样不断迭代，路径长度单调下降，且始终保持在自由空间内。
- **时间参数化**。几何路径只回答“走哪条线”，“多快走完”是另一个问题。沿路径的**弧长**套一条 3.3 中的梯形速度剖面，就把纯几何的折线变成了满足速度/加速度上限的时间轨迹。这就是机器人里标准的**路径-速度解耦**：路径规划只关心几何可行性，时间参数化只关心动力学可行性，两者互不干扰。

四、深度强化学习

4.1 强化学习的根本目标是什么（作业题 1）

一句话：在只能通过与环境交互获得标量奖励的前提下，学到一个策略，使长期累积奖励的期望最大。用数学写出来是

$$\pi^* = \arg \max_{\pi} \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^T \gamma^t r_t \right] \quad (25)$$

其中 $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots)$ 是一条轨迹，由策略 π 与环境共同生成； $\gamma \in [0, 1)$ 是折扣因子，用来保证无穷时域下求和的收敛，同时体现“近期奖励更重要”。

这句话里“只能通过交互”和“标量奖励”两个限定是关键，它们决定了强化学习与监督学习的三点本质差别：

- **没有标准答案，只有评价。** 监督学习每个样本都带正确标签，损失函数直接告诉模型“应该输出什么”；强化学习只得到“刚才这步值多少分”，并不知道“正确的动作应该是什么”。信用分配问题（credit assignment）由此而来：一个得分的动作未必是好动作，可能是它之后的动作好。
- **数据分布由策略自己决定。** 策略一变，采样到的状态分布随之改变，于是训练数据不再来自固定分布，SGD 的 i.i.d. 假设被破坏。这既是强化学习不稳定的根本原因，也解释了为什么 DQN 要用经验回放来打散样本相关性、为什么 PPO 要做重要性采样来校正新旧策略的分布差。
- **奖励可以稀疏且延迟。** 一局棋的胜负要到终局才知道，若直接用回报去更新，中间每一步的信用都极难分配。为此引入价值函数 $V(s)$ 、动作价值 $Q(s, a)$ 与优势 $A(s, a) = Q(s, a) - V(s)$ 这一整套工具，把终局的稀疏信号逐步回传并摊到中间每一步——这正是 4.2 中 Bellman 方程的意义。

4.2 马尔可夫决策过程与 Bellman 方程

强化学习问题的标准形式是马尔可夫决策过程（MDP），用五元组 (S, A, P, R, γ) 描述：状态集、动作集、转移概率、奖励函数、折扣因子。“马尔可夫”意味着下一状态只依赖当前状态与动作，而与更早的历史无关——这条假设使得价值函数可以写成递归形式。

定义策略 π 下的状态价值与动作价值（前者以“从状态 s 出发”为条件，后者以“从 (s, a) 出发”为条件）：

$$V^{\pi}(s) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \right], \quad Q^{\pi}(s, a) = \mathbb{E}_{\pi} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \right] \quad (26)$$

对它们做一步展开（利用马尔可夫性），就得到 Bellman 期望方程：

$$V^\pi(s) = \sum_a \pi(a|s) \sum_{s'} P(s'|s, a) [r + \gamma V^\pi(s')] \quad (27)$$

这个递归式是整个强化学习的枢纽：**它把一个无穷时域的优化问题变成了一个可以迭代求解的不动点问题**。对最优策略取 max 而不是对 π 求平均，得到 Bellman 最优方程：

$$V^*(s) = \max_a \sum_{s'} P(s'|s, a) [r + \gamma V^*(s')], \quad Q^*(s, a) = \mathbb{E} \left[r + \gamma \max_{a'} Q^*(s', a') \right] \quad (28)$$

后一条式子尤其重要：它给出了 Q 的**自洽条件**，而所有基于值的方法（Q-learning、SARSA、DQN）本质上都在用各种手段去逼近这个不动点。由此也立刻得到一个贪心的最优策略：

$$\pi^*(s) = \arg \max_a Q^*(s, a) \quad (29)$$

注意这里的 max 带来的一个后果：**max 算子是非线性的**，而它对估计误差是上偏的——若 Q 的估计带有噪声，取 max 之后期望值会系统性高于真实最大值。这一点是后面 DQN 需要 Double DQN 修正、以及 PPO 这类方法更偏爱策略梯度路线的深层原因。

4.3 DQN 和 Q-learning 的核心区别是什么（作业题 2）

先明确一点：**两者优化的是同一个目标**，即上面那条 Bellman 最优方程。差别完全在“Q 函数用什么表示”，以及由此带来的训练技巧。

Q-learning 是表格法。它把 Q 存成一张表，一格对应一个状态-动作对，每交互一步做一次更新：

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right] \quad (30)$$

有限 MDP 且每个状态-动作对被访问无穷多次时，只要学习率满足经典步长条件，该迭代以概率 1 收敛到 Q^* 。它的两个特点是天然成立的：**off-policy**（更新用的是 max，与行为策略无关，所以可以边探索边学最优策略），以及**每次只更新被访问的那一格**（表格里各格互不影响，因此不存在“一个更新污染全体”的问题，也不需要回放和冻结目标）。

DQN 用神经网络代替表格。 $Q(s, a; \theta)$ 由一个网络给出，损失是最小化自举目标与预测值的平方误差：

$$L(\theta) = \mathbb{E}_{(s, a, r, s') \sim D} \left[\left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta) \right)^2 \right] \quad (31)$$

换成函数逼近之后，原本“免费”的两条性质立刻消失，并且出现了所谓的 **deadly triad**——函数逼近、自举（bootstrap）、off-policy 三者同时出现时，训练可能发散。逐条看：

- **样本强相关**。表格法的更新是局部且独立的，样本顺序无所谓；而网络的每次梯度更新会影响整个参数曲面，于是相邻时间步的高度相关样本会让梯度沿着轨迹方向累积，破坏 SGD 的 i.i.d. 假设。**补救：经验回放**（experience replay）——把交互数据存进一个大缓冲区，训练时随机采样小批量，从而打散时间相关性，并让同一个样本被多次利用。
- **自举目标不稳定**。表格法的目标是 $r + \gamma \max Q(s', a')$ ，而 $Q(s', a')$ 存在另一格里，更新 (s, a) 不会改变它；但网络的目标里 $\max Q(s', a'; \theta^-)$ 与预测 $Q(s, a; \theta)$ 共享参数，于是“追着自己的尾巴跑”。**补救：目标网络**（target network）——另存一份参数 θ^- 专门算目标，每隔若干步才同步一次，让目标在短时间内是个“准静止”的靶子。
- **max 的高估**。4.2 提到 max 对噪声上偏，在函数逼近下这种上偏会被自举不断放大。**补救：Double DQN**——用在线网络选动作、用目标网络估值，把“选择”与“评估”解耦，显著减轻过估计。

一句话概括区别：**Q-learning 把状态空间当作可枚举的，DQN 用参数化函数把状态空间推广到连续、高维；代价是必须额外引入回放、目标网络等机制来对抗函数逼近带来的不稳定。**这也直接引出 DQN 相对表格法的一条优势（不必手工离散化，从而避开维度灾难与离散化误差）和一条劣势（没有收敛保证，训练过程可能振荡甚至崩溃）。

对比项	Q-learning（表格）	DQN（函数逼近）
Q 的表示	一张表，一格一个状态-动作对	神经网络 $Q(s, a; \theta)$
状态空间	必须离散化，维度指数增长	可直接吃连续、高维观测
单次更新影响	只影响被访问的那一格	影响整个参数化曲面
经验回放	不需要	必需，打散样本时间相关性
目标网络	不需要（目标项本身是独立的）	必需，冻结自举目标
收敛性	有限 MDP 下收敛到 Q^*	无保证，可能振荡或发散

表 2：Q-learning 与 DQN 的核心区别。两者目标函数相同，差别全在表示方式以及为对抗函数逼近不稳定而引入的机制。

4.4 策略梯度与 Actor-Critic

基于值的方法先学 Q 再取 $\arg\max$ ，动作空间必须是离散的（否则 $\max$ 难求）。另一条路线是**直接对策略参数化** $\pi_\theta(a|s)$ 并把目标 $J(\theta)$ 关于 θ 求梯度，这样动作可以是连续的，也更容易处理随机策略。

利用轨迹概率的对数导数技巧，可以推出策略梯度定理：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \Psi_t \right] \quad (32)$$

其中 Ψ_t 是“这一步有多好”的度量。它取什么，就决定了算法的名字：

- **REINFORCE**: $\Psi_t = G_t$ ，即整条轨迹的折扣回报（蒙特卡洛）。它**无偏**（因为完全不引入任何估计），但**方差很大**—— G_t 包含了整条轨迹后续所有随机性，其中大部分与当前动作无关。此外它是纯 on-policy 的：每条轨迹用完即弃，样本效率极低。
- **带基线的 REINFORCE**: $\Psi_t = G_t - b(s_t)$ 。可以证明减掉一个只依赖状态的基线**不引入偏差**：因为

$$\mathbb{E}_{a \sim \pi} [\nabla_{\theta} \log \pi_{\theta}(a|s)] = 0 \quad (33)$$

($\nabla \log \pi$ 对动作的求和为零)，所以任何与动作无关的量 $b(s)$ 乘以它，期望都是零。而取 $b(s) = V(s)$ 时**方差最小**（可以逐状态求导验证），因为 $V(s)$ 恰好是 G_t 的条件期望，减掉它就等于去掉了“初始状态好不好”这部分与动作无关的波动。

- **Actor-Critic**: 用一个评论家网络 $V_{\phi}(s)$ 去逼近基线，并用**单步自举**构造优势：

$$\hat{A}_t = r_t + \gamma V_{\phi}(s_{t+1}) - V_{\phi}(s_t) \quad (34)$$

这就是“演员-评论家”这个名字的由来：演员（策略网络）负责选动作，评论家（价值网络）负责给动作打分。用单步 TD 误差代替蒙特卡洛回报，方差进一步降低，但引入了偏差（因为 V_{ϕ} 本身是估计值），这就是**偏差-方差权衡**。

把这个权衡一般化，就是**广义优势估计 (GAE)**：用参数 λ 在两者之间插值，递归形式为

$$\hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1}, \quad \delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \quad (35)$$

$\lambda = 0$ 退化为单步 TD（方差小、偏差大）， $\lambda = 1$ 退化为蒙特卡洛优势（无偏、方差大），实践中常取 $\lambda \approx 0.95$ 。另需注意一个工程细节：**自举必须在真正终止处切断**。截断（truncation，例如达到时间上限）不是任务结束，仍应继续自举；若把两者混为一谈，价值会被系统性低估。

4.5 为什么 PPO 比 REINFORCE 更稳定（作业题 3）

REINFORCE 的不稳定有两个层次，两者都不在“学习率没调好”这个层面。

第一层：梯度估计的方差大。如上所述，用整条轨迹的回报 G_t 做权重时，其中与当前动作无关的波动会原样进入梯度。加基线可以缓解，但只要还是蒙特卡洛，方差就仍然可观。

第二层（更致命）：更新步长不受控。REINFORCE 每次更新后策略可以任意远地移动。而它又是 on-policy 的——一旦策略被推坏，接下来采到的数据全部来自坏策略，梯度方向随之变坏，形成正反馈：**策略崩溃** → **数据变坏** → **更新更坏**。这是结构性问题，靠调小学习率只能延缓、不能消除。

PPO 用两个机制同时处理这两层。

机制一：重要性采样 + 裁剪。先写出一批数据上策略梯度目标的重要性采样形式。记概率比 $r_t(\theta) = \pi_\theta(a_t|s_t) / \pi_{\theta_{\text{old}}}(a_t|s_t)$ ，则同一批数据可以被复用多次而不偏。但 r_t 一旦偏离 1 太远，估计的方差会爆炸，所以 PPO 用 min 与 clip 构造一个“悲观”的替代目标：

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t [\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \varepsilon, 1 + \varepsilon) A_t)] \quad (36)$$

这个式子的作用要分情况看，关键在于 min 与 clip 的配合：

- **优势为正 ($A_t > 0$ ，这是个好动作) 时：**目标随 r_t 线性增长（鼓励提高其概率），但一旦 $r_t > 1 + \varepsilon$ ，裁剪项变成常数 $(1 + \varepsilon)A_t$ ，而 min 会选中它，于是目标对 θ 的导数变为零——梯度被直接掐断，策略不能再继续提高这个动作的概率。
- **优势为负 ($A_t < 0$ ，这是差动作) 时：**目标随 r_t 减小而增大（鼓励降低其概率），一旦 $r_t < 1 - \varepsilon$ ，裁剪项再次成为 min 的取值，导数同样归零——也不能继续压低它的概率。
- **r_t 在 $[1 - \varepsilon, 1 + \varepsilon]$ 内时：**不裁剪与裁剪两支相等，目标退化为普通的重要性采样策略梯度，梯度正常。

也就是说，**裁剪把“一次更新允许的策略变化量”硬性限住了**：只有让概率比落在区间内的样本才会产生梯度，跑出去的样本贡献为零。这正是“近端”（proximal）一词的含义——要求新策略只能待在旧策略的近邻。

作为对照，去掉裁剪 ($\varepsilon \rightarrow \infty$) 后目标变成纯的重要性采样估计，同一批数据训练多个 epoch 时概率比会不断偏离 1，目标值随 r_t 无界增长，等价于对高 r_t 样本给了无上限的权重，训练会在几次更新内被推离旧策略很远。

机制二：GAE(λ) 优势估计。如 4.4 所述，用 λ 在偏差与方差之间取平衡，比纯蒙特卡洛回报的信号更干净。

两个机制合起来的效果可以定量描述：**用新旧策略之间的 KL 散度 $\text{KL}(\pi_{\theta_{\text{old}}} \parallel \pi_\theta)$ 作为“策略走了多远”的度量**，裁剪的作用就是把每次更新的这个量压在一个很小的常数附近，而它并不依赖学习率多大——即使学习率相同，不裁剪的版本也会把策略推得远得多。这就是“PPO 比 REINFORCE 稳定”的完整解释：稳定性来自**对更新幅度的结构性约束**，而不是来自更保守的超参数。

顺带说明两个工程细节。其一，实现里通常在裁剪之外再加一道保险：若本次更新的平均近似 KL 超过目标值的 1.5 倍，就提前结束本轮 epoch，不再继续用这批数据。其二，重要性采样要求同一条轨迹在训练时使用的**旧策略 log 概率必须被记录并冻结**，否则 r_t 会被算错，裁剪也就失去意义。

4.6 A3C 算法的主要优势是什么（作业题 4）

A3C(Asynchronous Advantage Actor-Critic)在 A2C 的基础上引入了**异步并行**：多个 worker 各自在环境副本里独立采样、独立算梯度，异步地更新到共享参数上。它带来三个优势，逐条看。

优势一：降低梯度估计的方差。设每个 worker 给出的梯度估计 g_i 相互独立、方差同为 σ^2 ，那么平均 B 个 worker 的估计的方差为

$$\text{Var} \left[\frac{1}{B} \sum_{i=1}^B g_i \right] = \frac{\sigma^2}{B} \implies \text{std} \propto \frac{1}{\sqrt{B}} \quad (37)$$

即方差按 $1/B$ 下降、标准差按 $1/\sqrt{B}$ 下降。这是并行采样最直接、也是最可量化的收益：多开几个环境，相当于在不改算法的情况下把梯度噪声按指数级压下去。注意这条结论只要求各 worker 的估计**独立**，并不要求同分布，所以各 worker 跑不同初始状态、不同超参也是成立的。

优势二：样本天然去相关，省掉经验回放。DQN 靠一个巨大的回放池来打散样本的时间相关性，代价是内存占用（池子往往要几十万到上百万条）和采样开销。A3C 的多个 worker 处于不同的环境状态，它们产生的数据本身就互不相关，于是可以像 on-policy 方法一样“用完即弃”，不需要维护回放池。

优势三：没有同步屏障，硬件利用率和吞吐更高。同步方法（如 A2C）必须等所有 worker 都跑完一批才能做一次更新，整批的耗时取决于最慢的那个 worker；A3C 中谁先跑完谁先更新，参数始终在被推进，避免了同步等待，在多机异构环境下吞吐明显更高。

代价与后续发展。异步带来的不只是好处。由于各 worker 用的是不同时刻的参数，算出的梯度相对于当前参数是**滞后**的，这会引入额外的偏差，且“不同 worker 提交梯度的时刻不同”使得训练过程难以复现。实践中，在算力充足时 A2C（同步平均梯度）往往能达到同等甚至更好的效果，而且结果可复现、更容易调试。这也解释了为什么后来的主流（包括 PPO 的常见实现）多采用同步并行采样，而不是严格意义的异步更新。

五、小结

把三层内容放在一起看，可以总结出三条贯穿始终的思想。

第一，合适的表示能把复杂运算变成标准运算。齐次矩阵把“旋转加平移”变成一次矩阵乘法，于是链式复合、求逆、作用到点全部成为线性代数里的标准操作；四元数把姿态复合变成四元数乘法，且全程无奇异；MDP 的马尔可夫性把无穷时域的优化变成 Bellman 不动点迭代。这些“换表示”的动作，其价值远大于记住具体公式。

第二，约束要在正确的位置表达。轨迹规划里，如果只给端点条件，中间段的峰值就完全不受控（三次多项式的峰值是 $1.5\Delta q/T$ 与 $6\Delta q/T^2$ ，随时长缩短而迅速超限）；而把限制直接写进剖面结构（LSPB 的分段常值），或者写成显式的可行性判据（ $T \geq \Delta q/v_{\max} + v_{\max}/a_{\max}$ ），问题就变成可判定的。强化学习里同理：不约束更新幅度，策略就会跑飞；把约束以裁剪的形式写进目标函数，训练就稳定了。

第三，稳定性问题几乎都可以归结为“方差”与“步长”。策略梯度用蒙特卡洛回报导致方差大，减一个只依赖状态的基线可以无偏地降方差（因为 $\nabla \log \pi$ 对动作积分为零）；单步自举降方差但引入偏差，GAE 的 λ 就是在两者之间调；并行采样把方差按 $1/B$ 压下去；而 PPO 的裁剪则从另一个方向解决问题——不降方差，而是直接限制每次更新能把策略推多远。把这几个维度想清楚，强化学习里大部分算法的设计动机就都串起来了。

有待进一步理解的问题：

- 裁剪之外，PPO 还有 KL 惩罚、自适应 KL 系数等变体，它们在什么条件下更优、代价是什么；
- 从“在仿真里学会”到“在真机上可用”之间的差距（sim-to-real）：观测延迟、执行器动力学、接触模型误差分别如何影响策略；
- 把轨迹规划的先验（例如 RRT 生成的示范轨迹）作为强化学习的 warm-start，在理论上如何影响样本复杂度。